LEARN CODING

ale66

github.com/ale66/learn-coding

STRING METHODS

WHY SPLIT?

Real data rarely arrives in the shape we need

- a log line: `'2026-10-29 ERROR disk full'`
- a CSV row: `'Milan,1.4,Italy'`
- a chat message: `'ciao a tutti, buona giornata'`
- a file path: `'data/raw/channel_01.json'`

Splitting turns **one string** into **a list of pieces** we can count, filter and analyse.

Tokenising a message is the first step of almost every NLP pipeline.

A REVIEW OF METHODS

Python variables (later, *objects*) come with built-in functions attached.

```
1 mylist = []
```

now `variable_name.method_name(parameters)` works

```
1 mylist.append('the first')
2 mylist.append('the second')
3 mylist.append('and the third')
4
5 l = len(mylist)
6
7 print(mylist[2:])
```

```
['and the third']
```

SPLITTING METHODS

mystring.split()

```
1 mytext = "To be or not to be"
2
3 mywords = mytext.split()
4
5 print(mywords)
```

```
['To', 'be', 'or', 'not', 'to', 'be']
```

- `split()` returns a **list of strings**
- by default, we split on white spaces
- the original string is **not modified** (strings are immutable)

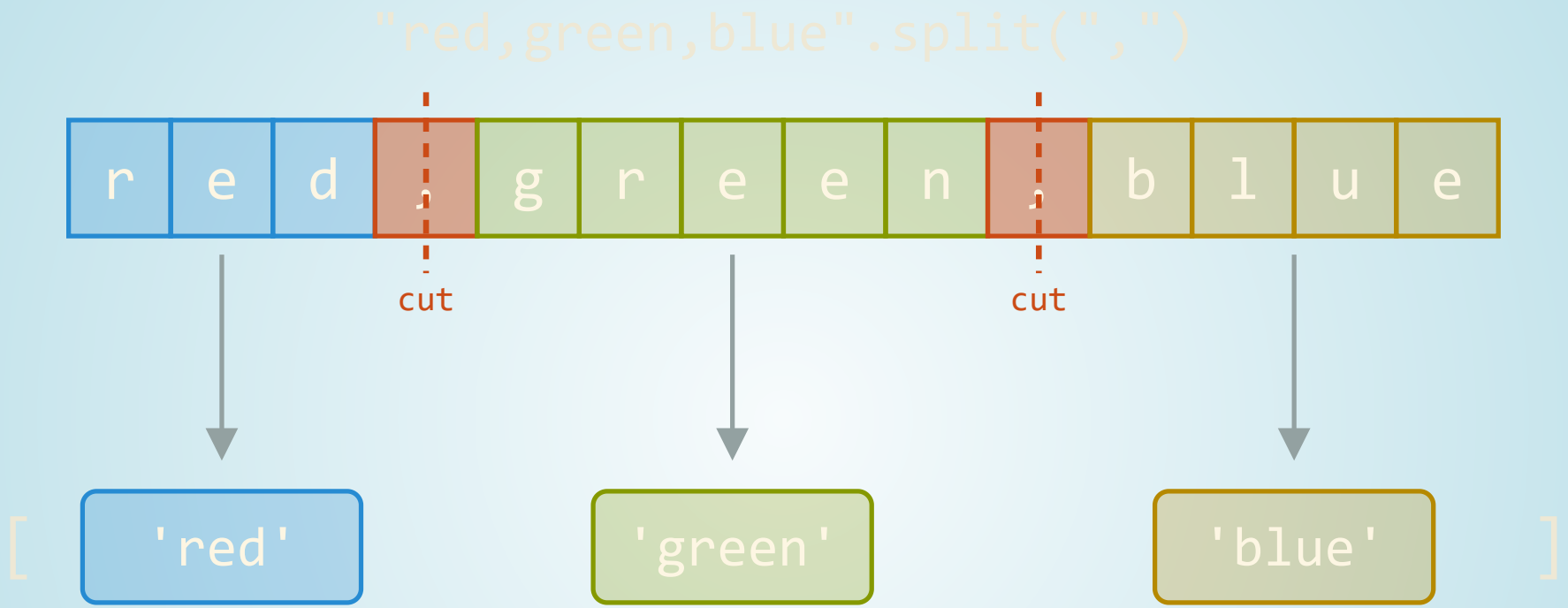
White spaces are gone!

```
1 mytext = '''Two households, both alike in dignity,
2           In fair Verona, where we lay our scene,'''
3
4 mywords = mytext.split(',')
5
6 print(mywords)
```

```
['Two households', ' both alike in dignity', '\n          In fair Verona', ' where we lay our scene', '']
```

Commas are gone

STRING SLICING



the separator is consumed: it does not appear in the result

separators are cut out

PUNCTUATION AND LINE BREAKS

```
1 mytext = '''Two households, both alike in dignity,
2           In fair Verona, where we lay our scene,'''
3
4 mywords = mytext.split()
5
6 print(mywords)
```

```
['Two', 'households,', 'both', 'alike', 'in', 'dignity,', 'In', 'fair', 'Verona,',
'where', 'we', 'lay', 'our', 'scene,']
```

PARAMETERS FOR `.split()`

```
1  mytext.split(sep=None, maxsplit=-1)
```

parameter	meaning	default
`sep`	the delimiter to cut on	`None` (whitespace)
`maxsplit`	maximum number of cuts	`-1` (no limit)

GIVING A SEPARATOR

```
1 myrowofdata = "Milan,1.4,Italy"
2
3 myrowofdata.split(',')
```

```
['Milan', '1.4', 'Italy']
```

```
1 myrowofdata = "Milan,1.4,Italy"
2
3 myrowofdata.split(sep=',')
```

```
['Milan', '1.4', 'Italy']
```

Find the `.doc` file and make it a `.docx` file

```
1 myfilepath = 'C:/Users/aless/Downloads/_git/learn_code/58-string-methods/myfile.do
```

```
1 the_pieces = myfilepath.split('/')
2
3 print(the_pieces)
```

```
['C:', 'Users', 'aless', 'Downloads', '_git', 'learn_code', '58-string-methods',
'myfile.doc']
```

the file name is the last element of the path

```
1 the_file_name = the_pieces[-1]
2
3 print(the_file_name)
```

```
myfile.doc
```

chop off the `.doc` and replace it

```
1 two_pieces = the_file_name.split('.')
2
3 print(two_pieces)
```

```
['myfile', 'doc']
```

keep the actual name (first ele. of the list) and change the suffix

```
1  new_name = two_pieces[0] + '.docx'
2
3  print(new_name)
```

```
myfile.docx
```

```
1  new_name = two_pieces[0] + '.docx'
2
3  new_file = the_pieces.pop() + new_name
4
5  print(new_file)
```

```
myfile.docmyfile.docx
```

what happened to the /s?

JOIN, THE ANTI-SPLIT

THE `.join` METHOD

Dual to `.split`, with a funny syntax

```
1 prep_list = ['the first', 'the second', 'and the third']
2
3 mystring = ''.join(prep_list)
4
5 print(mystring)
```

```
the firstthe secondand the third
```

```
1 prep_list = ['the first', 'the second', 'and the third']
2
3 # comma followed by a whitespace
4 mystring = ', '.join(prep_list)
5
6 print(mystring)
```

```
the first, the second, and the third
```

SPLIT+JOIN IS THE KEY TO STRING PROCESSING

```
1 words = ['to', 'be', 'or', 'not', 'to', 'be']
2
3 mysentence = ' '.join(words)
4
5 print(mysentence)
```

```
to be or not to be
```

Quiz: what does it print?

```
1 mytest = 'a,b,c'
2
3 recombined = ','.join(mytest.split(","))
4
5 print (recombined)
```

Quiz: what are getting?

```
1 test = 'a,b,c'
2
3 recombined = ','.join(test.split(","))
4
5 print (recombined)
```

```
a,b,c
```

FOR REFERENCE:
SPLITTING IN PANDAS

PANDAS DATAFRAMES

```python
import pandas as pd

df = pd.DataFrame({"channel": ["it_news/2026", "it_sport/2025"]})

df[["name", "year"]] = df["channel"].str.split("/", expand=True)
```

channel	name	year
it_news/2026	it_news	2026
it_sport/2025	it_sport	2025

`expand=True` turns the pieces into **columns**.

SPLITTING LISTS, INSTEAD

Slice a **list** into chunks of size `n`:

```
1  data = list(range(10))
2  n = 4
3
4  chunks = [data[i:i + n] for i in range(0, len(data), n)]
```

SUMMARY

- `split` goes from text to list, `join` does the opposite
- `split(" ")` and `split()` behave differently
- `split` does not change the original string
- deploy `strip()` when lines carry trailing spaces or newlines
- [for CSV files use specific line parsers in `csv` or `pandas`]